Travaux Pratiques
Programmation Multi-Paradigme
Licence 3 Informatique

Julien BERNARD et Arthur HUGEAT

Table des matières

Projet n°1 : Sérialisation dans un fichier **3**
Étape 1 : Fichier binaire 3
Étape 2 : Opérateurs de sérialisation 3
Exemple d'utilisation 3

Projet n°2 : Unités et quantités **5**
Étape 1 : Unités 5
Étape 2 : Quantités 5
Étape 3 : Opérateurs littéraux 6
Exemple d'utilisation 6

Projet n°3 : Signals **7**
Étape 1 : Signal 7
Étape 2 : Combiner 7
Exemple d'utilisation 9

Consignes communes à tous les projets

Au cours de cette UE, vous avez **trois** projets à réaliser à raison d'un projet pour deux séances de trois heures de travaux pratiques encadrées. Les projets sont à faire et à rendre dans l'ordre du présent sujet. Vous avez le droit de vous mettre en binôme, dans ce cas vous devez préciser vos deux noms dans un fichier à la racine de vos rendus. Les projets faits en binôme ou en monôme seront évalués de la même manière.

Pour chaque projet, vous devrez implémenter une interface donnée dans un fichier d'en-tête, ainsi qu'un ensemble de tests unitaires pour cette interface. Les tests serviront à montrer que votre implémentation est correcte et complète.

Les sources doivent être écrites par vous-même, l'utilisation d'IA génératrice (ChatGPT, Copilot, Claude, ...) est donc strictement interdite. Si vous ré-utilisez un bout de code provenant d'un site web (Stack Overflow par exemple), vous devez mettre en commentaire le lien de votre source.

Il est attendu que vos codes sources et les commentaires soient rédigés en anglais et uniquement en anglais.

Projet n°1 : Sérialisation dans un fichier

Le but de ce projet est d'implémenter des classes pour lire et écrire des fichiers binaires, pour réaliser une bibliothèque de sérialisation. La sérialisation consiste à stocker des données en format binaire portable. Elle est utilisée pour la sauvegarde de fichiers, les protocoles réseau, etc.

Étape 1 : Fichier binaire

Dans cette étape, vous devrez implémenter deux classes, `OBinaryFile` pour les fichiers en écriture, et `IBinaryFile` pour les fichiers en lecture. Comme il s'agit de classes représentant des ressources, il sera nécessaire d'appliquer la *Rule of Five*.

L'implémentation de ces deux classes utilisera l'API C qui se trouve dans `<cstdio>`, il est donc formellement interdit d'utiliser toute autre API.

Pour permettre une meilleure compatibilité, le format du fichier binaire doit respecter ces règles :

1. Les données sont stockées dans l'ordre d'insertion : *First In, First Out (FIFO)*
2. Les collections d'objets doivent stocker d'abord le nombre d'éléments puis les objets dans le même ordre que leur itérateur.

Durant le développement de votre projet, vous allez avoir besoin de vérifier les données stockées dans vos fichiers. Les IDE modernes proposent généralement une vue binaire mais vous pouvez également utiliser les commandes `xxd` ou `hexdump` pour afficher les données du fichier en format hexadécimal.

Étape 2 : Opérateurs de sérialisation

Dans cette étape, vous utiliserez les deux classes définies précédemment pour sérialiser et désérialiser un ensemble de types de base en surchargeant les opérateurs `<<` et `>>`. Attention, le format de sérialisation devra être en *big endian* exception faite des flottants (`float` et `double`). On ne prendra pas en charge les pointeurs et les références.

Puis vous aurez à sérialiser des types conteneurs de la bibliothèque standard de manière générique.

Exemple d'utilisation

```
#include <cassert>
#include "Serial.h"

struct Foo {
  int16_t i;
  double d;
  std::string s;
};

serial::OBinaryFile& operator<<(serial::OBinaryFile& file,
    const Foo& foo) {
  return file << foo.i << foo.d << foo.s;
```

```
}

serial::IBinaryFile& operator>>(serial::IBinaryFile& file,
    Foo& foo) {
  return file >> foo.i >> foo.d >> foo.s;
}

int main() {
  Foo foo;
  foo.i = 42;
  foo.d = 69.0;
  foo.s = "Hello";

  {
    serial::OBinaryFile out("foo.bin");
    out << foo;
  }

  struct Foo copy;

  {
    serial::IBinaryFile in("foo.bin");
    in >> copy;
  }

  assert(foo.i == copy.i);
  assert(foo.d == copy.d);
  assert(foo.s == copy.s);
}
```

Projet n°2 : Unités et quantités

Le but de ce projet est d'implémenter une bibliothèque de mesures physiques qui permet de coder les unités du système international. Nous tirons profit du C++ moderne pour offrir à l'utilisateur final une écriture simple et intuitive de formule physiques.

Ce projet permettra de travailler le *Template Metaprogramming* et comme vous le verrez très vite, vous aurez énormément d'erreurs de compilation. Il pourra vous être utile de changer de compilateur pour avoir des messages plus explicites mais l'évaluation de votre code sera faite avec `gcc` donc pensez à bien vérifier votre implémentation finale avec lui.

Pour changer de compilateur, vous devez installer un autre compilateur sur votre système comme par exemple `clang`. Ensuite vous devez re-générer votre projet cmake grâce à cette commande :

```
C=clang CXX=clang++ cmake -B build-clang
```

Il ne vous reste plus qu'à compiler votre projet comme vous avez l'habitude :

```
cmake --build build-clang
```

Dernière petite précision, pour implémenter ce projet vous aurez probablement besoin de définir des fonctions, classes ou structures utilitaires. Par convention, on place ces définitions dans un namespace imbriqué qu'on appelle `details`.

Étape 1 : Unités

Dans le squelette de code, vous trouverez la structure templatée `Unit`. Elle permet de représenter toutes les unités du système international, vous n'avez donc pas à la modifier. Le système d'unités international définie sept unités de base : la seconde, le mètre, le kilogramme, l'ampère, le kelvin, la mole et la candela. Le produit de ces unités de base permet de dériver d'autres unités comme par exemple une vitesse, une force, une fréquence...

La première étape de ce projet sera donc de comprendre comment marche la structure `Unit` et de l'utiliser pour exprimer les unités qui vous sont demandées. Commencez par les unités de base avant d'essayer de définir les unités dérivées.

Il est fortement recommandé de lire la documentation de `std::ratio` dont vous aurez besoin dans la suite et qui peut vous servir d'inspiration pour cette partie du projet.

Étape 2 : Quantités

La deuxième partie consiste à implémenter une classe `Qty` représentant des quantités. Les quantités sont représentées par un entier, associé à une unité et un ratio. Par exemple une quantité représentant des millimètres aura comme type `Qty<Metre, std::milli>`. Une quantité sera initialisée à une valeur nulle ou alors on pourra lui passer une valeur.

La difficulté ici consiste à implémenter correctement les opérateurs de comparaisons et les opérateurs arithmétiques, en prenant en compte les unités mais aussi les ratios.

Grâce à cette représentation, il est possible d'exprimer les unités de distance du système impérial (Mile, Yard, Foot, Inch). Il n'est pas nécessaire d'ajouter des nouvelles unités car on peut les exprimer comme des quantités de distance. Suivant la même logique, vous pouvez définir le nœud (Knot) comme une quantité de vitesse.

Toutes les constantes utilisées pour définir des quantités doivent être définies avec huit chiffres significatifs.

Étape 3 : Opérateurs littéraux

En C++, on peut définir des opérateurs littéraux personnalisés. Par exemple, la bibliothèque standard vous permet de définir un nombre complexe de cette façon :

```
// the suffix i will call the function
// operator""i(long double arg) to create a complex number
std::complex<double> c = 1.0 + 1i;
```

La dernière partie consiste à ajouter des opérateurs littéraux pour le déclaration des quantités. Vous devez donner l'implémentation pour les opérateur littéraux des unités de base du système d'unités international et également l'expression des degrés Celsius.

Exemple d'utilisation

```
#include "Units.h"

#include <cassert>

int main() {
  using namespace phy::literals;

  auto speed = 100000_metres / 3600_seconds; // 100 km/h
  assert(decltype(speed)::Unit::metre == 1);
  assert(decltype(speed)::Unit::second != 1);

  phy::Qty<phy::Metre, std::milli> mm(32);
  auto nm = phy::qtyCast<phy::Qty<phy::Metre, std::nano>>(mm);

  return 0;
}
```

Projet n°3 : Signals

Le but de ce projet est d'implémenter un système d'émission de données et de collecte de résultats. Vous allez devoir implémenter une classe qui sera chargée d'enregistrer des fonctions et de les appeler ainsi qu'un ensemble d'objets vous permettant de traiter l'ensemble de ces résultats. Ce type d'interface est souvent utilisé dans des projets en C++ comme par exemple dans la bibliothèque d'interface utilisateur Qt [1].

Vous allez donc travailler deux aspects du langage :
- Les templates (et la spécialisation de template)
- La programmation fonctionnelle

Étape 1 : Signal

Cette classe est chargée de stocker et d'appeler les fonctions de rappel (souvent appelées *slot* ou *callback*). On les enregistre d'abord grâce à la méthode `connectSlot()`, puis on émet un signal, grâce à la méthode `emitSignal()`, qui va appeler et transmettre les paramètres à toutes les fonctions enregistrées.

Un objet du type `Signal` peut être créé à partir d'un `Combiner` ou alors en passant directement les paramètres à donner au constructeur du `Combiner`. Le fonctionnement des `Combiner` sera détaillé dans la prochaine étape.

L'appel à la méthode `connectSlot()` enregistre donc une fonction de rappel et renvoie un identifiant unique. Cet identifiant vous permettra de désinscrire la fonction de rappel grâce à la méthode `disconnectSlot()`. La nature de cet identifiant est opaque, c'est à dire que vous pouvez le définir comme bon vous semble avec la seule contrainte que chaque identifiant ne doit correspondre qu'à une seule fonction pour une instance donnée de `Signal`.

La méthode `emitSignal()` permet d'appeler toutes les fonctions enregistrées. Les paramètres passés à la méthode doivent être donnés aux fonctions de rappel et leurs résultats sont donnés au `Combiner` afin qu'il les traite et qu'il donne la valeur à retourner à l'issue de la méthode.

Une grosse difficulté pour pouvoir travailler avec cette classe sera d'être capable d'extraire le type de retour des fonctions de rappel ainsi que les types de leurs paramètres. Pour ce faire, vous pouvez étudier la déclaration de `std::function` pour comprendre comment faire.

Étape 2 : Combiner

Un `Combiner` est un objet qui doit appliquer un traitement sur chaque résultat des fonctions de rappel et qui retourne la synthèse de ces résultats. Contrairement à ce que vous auriez l'habitude en programmation orientée objet, ces `Combiner` n'auront pas d'ancêtre commun mais ils devront tous comporter ces éléments :
- Une méthode `combine()` qui prendra le résultat de chaque fonction de rappel
- Une méthode `result()` qui renverra le résultat final, après que tous les appels à `combine()` aient été faits, lors de l'appel à `Signal::emitSignal()`

1. https://www.qt.io/development/qt-framework

— Un alias de type `result_type` qui représente le type retourné par la méthode `result()` et qui sera également du même type que le type retourné par `Signal::emitSignal()`

Ce choix d'architecture peut vous paraître étrange mais, en plus de l'intérêt pédagogique, vous verrez qu'en fonction du type de `Combiner` le type de retour de `result()` sera différent. Il aurait donc été impossible de gérer avec un héritage classique ce changement de type.

Dans ce projet, on vous demande de fournir un nombre fini de `Combiner` qui auront chacun un traitement spécifique :

— `DiscardCombiner` qui va écarter tous les résultats des fonctions de rappel
— `LastCombiner` qui ne va conserver que le dernier résultat, c'est-à-dire le résultat de la dernière fonction de rappel enregistrée dans le signal
— `VectorCombiner` qui va stocker tous les résultats dans un `std::vector` dans le même ordre que l'enregistrement des fonctions de rappel
— `PredicateCombiner` qui va appliquer un prédicat unaire ou binaire et qui renverra le dernier résultat à valider ce prédicat ou `std::nullopt` si aucune valeur ne valide le prédicat. Le prédicat sera passé au constructeur de l'objet et ne pourra pas être changer après. Petite précision, dans le cas d'un prédicat binaire, le prédicat devra être appliqué entre la précédente valeur valide et la nouvelle valeur retournée par la fonction de rappel

Attention, les fonctions de rappel peuvent ne rien retourner. Dans ce cas, les `Combiner` ne reverront rien mais votre code doit pouvoir compiler.

Exemple d'utilisation

```
#include <cassert>
#include <cstddef>
#include <cstdio>
#include "Signal.h"

void callback(int param) {
  std::printf("Hello %i\n", param);
}

int main() {
  // define a signal for functions that takes an int
  // and returns void
  sig::Signal<void(int)> sig;

  // connect a simple function
  sig.connectSlot(callback);

  // connect a lambda function
  int res = 0;
  sig.connectSlot([&res](int x) { res = x; });

  // emit the signal
  sig.emitSignal(1);

  // here res equals 1 and "Hello 1" is printed on stdout
  assert(res == 1);

  sig::Signal<int(int), sig::PredicateCombiner<int>> sig2([](int lhs, int rhs) -> bool {
    return lhs < rhs;
  });

  sig2.connectSlot([](int x) { return x;});
  sig2.connectSlot([](int x) { return x * x;});
  std::size_t id = sig2.connectSlot([](int x) { return x * -1;});

  auto result = sig2.emitSignal(8);
  assert(result); // has a value
  assert(*result == -8);

  sig2.disconnectSlot(id);
  result = sig2.emitSignal(8);
  assert(result); // has a value
  assert(*result == 8);
}
```